Payments, Campuses and the Director Portal

What changed across the web app, the API and the parent app — and everything needed to make it run.

The one idea behind all of it

The browser never tells us what was paid. A checkout is opened server-side for an amount we computed, and a payment is recorded only after PayChangu has been asked, over our own authenticated connection, what actually happened. A forged webhook achieves one thing: it makes us look up a reference and find it unpaid. This is the fix the PAYMENT INTEGRITY note in `firestore.rules` has been asking for.

1. What changed

PayChangu payments

Two kinds of money, one gateway, and they are not symmetrical. Fee amounts are chosen by the payer, because part-payment is how school fees are actually paid. The subscription amount is recomputed server-side from `services/pricing.js` and ignored if supplied — settling it marks the school active, so a client-set price would let a school subscribe for one kwacha.

File	Role
`api/_lib/paychangu.js`	Create checkout, verify, check webhook signature
`api/_lib/settle.js`	Verified transaction → the record it stands for. Idempotent
`api/_lib/firebaseAdmin.js`	Admin SDK (Auth + Firestore) for serverless functions
`api/payments/initiate.js`	POST — opens a checkout, returns the URL
`api/payments/webhook.js`	POST — PayChangu's callback
`api/payments/status.js`	GET — polled by the client; settles if the webhook is late

Why both a webhook and a poll. Either can arrive first and either can go missing. Both run the same settlement inside a Firestore transaction: whoever flips the status wins, the other finds it already done. A same-process lock would not work — they usually land on different serverless instances.

Campuses

One school, several sites. The school stays the unit of ownership; the campus becomes the unit of operation. Staff carry `allCampuses` or `campusIds[]`; a student belongs to exactly one campus and moves only by approved transfer, which clears their class placement because “Form 2A” exists on both sites.

- Setup now asks for campuses up front — splitting a school after enrolling 800 children means 800 approvals.
- Campuses are archived, never deleted: attendance, marks and payments all point at them.
- Records with no campus recorded count against the main campus, so no history vanishes.
- A single-campus school never sees a campus picker anywhere.

Director portal

A new role and portal at `/director`: school-wide, reads everything, edits almost nothing. Schools were handing directors an administrator login, which let them edit the very records they later sign off on. In `firestore.rules`, `isDirector()` is deliberately kept out of `isAdminOrHead()`; their one write on `/students` is narrowed by rule to the campus fields.

The parent app

The app used to write payments straight to Firestore, mint its own `RCP-…` receipt number and show “Payment Received” with a tick. The balance dropped. Nothing had checked that any money moved.

	Pay now	I have already paid
Path	PayChangu, in the device browser	Reported to the school
Writes	Nothing from the app — the webhook writes the receipt	status: unverified, **no receipt number**
Balance	Falls, once confirmed	Unchanged
Screen says	“Payment received · Receipt RCP-…”	“Sent to the school”

Reporting stayed, because a family who paid at a bank counter genuinely needs to tell the school. What went was the pretence that it was a receipt. Unconfirmed money now shows as its own line on the balance card — visible, so a report never looks lost; not subtracted, so nobody is told they owe less than they do.

2. Making it work

Step 1 — Environment variables (Vercel)

Project → Settings → Environment Variables, ticked for Production. None carry a `VITE_` prefix, so none reach the browser bundle.

Variable	Where it comes from
`PAYCHANGU_SECRET_KEY`	PayChangu dashboard → Settings → API Keys (sec-live-…)
`PAYCHANGU_WEBHOOK_SECRET`	Same screen — the webhook signing secret
`FIREBASE_ADMIN_PROJECT_ID`	Firebase service account
`FIREBASE_ADMIN_CLIENT_EMAIL`	Firebase service account
`FIREBASE_ADMIN_PRIVATE_KEY`	The full PEM, newlines written as \n
`APP_URL`	https://ssp-pep.vercel.app — builds the callback and return URLs

A full annotated list is in `.env.example` at the repo root. Optional: `PAYCHANGU_API_BASE`.

Step 2 — PayChangu dashboard

Set the webhook URL to:

```
https://ssp-pep.vercel.app/api/payments/webhook
```

Every checkout also sends its own callback and return URL built from `APP_URL`, so the dashboard setting is a backstop rather than the only path. Start with a `sec-test-…` key.

Step 3 — Deploy the rules, then the code

The new collections are refused until the rules are live, so deploy them first:

```
firebase deploy --only firestore:rules
git add -A
git commit -m "Add PayChangu payments, campuses and the director portal"
git push
```

Pushing to `main` triggers the Vercel deploy to ssp-pep.vercel.app. Do the env vars first — otherwise payments answer 503 until they exist.

Step 4 — The parent app

```
cd acalink
flutter pub get
flutter build apk --release
```

Two new packages (`http`, `url_launcher`) and two AndroidManifest entries: an explicit `INTERNET` permission and a `<queries>` intent for https VIEW. Without the second, Android 11+ hides every browser from url_launcher and the checkout opens onto nothing — a failure that only appears on API 30 and above.

To point a debug build at a local backend:

```
flutter run --dart-define=ACALINK_API=http://10.0.2.2:3000
```

Step 5 — Set the school up

- **Admin** → **first sign-in** now asks three questions: day/boarding, current term, and how many sites the school teaches from. Name the campuses here.
- **Admin** → **Campuses** to add or archive sites later. Adding a second one switches campus scoping on across the product.
- **Admin** → **User Management** → Role **Director** creates the director account. The campus picker appears there once a school runs more than one site.
- **Admin or Director** → **Campus Transfers** is where a student's move between campuses is approved.

3. Checking it works

- Bursar → Payments → **Collect Online** → pick a student → enter an amount.
- The checkout opens in a new tab; the dialog stays open and polls.
- On success a row appears in `payments` with `source: 'paychangu'`, `status: 'verified'` and a gatewayTxRef — and the balance moves.
- `payment_transactions/{txRef}` should read `success` with a settledAt and a paymentId.
- A transaction stuck at `pending` means the gateway has not confirmed it. Reopen the dialog: the status endpoint re-verifies on every poll, so a late confirmation settles itself.

Test suite: `npm run test:rules` — 100 tests, 26 of them new, covering campuses, transfer approvals and the "nobody can invent a payment" rules.

4. Two things needing your decision

Revoke the committed service-account key

`ssp-pep-firebase-adminsdk-fbsvc-a71809ca23.json` is tracked in the GitHub repository. That key grants full database and auth access. Ignore patterns have been added for future keys, but **.gitignore does nothing for an already-tracked file**, and it is in the history regardless. Revoke it in the Google Cloud console and issue a new one — untracking it now does not undo the exposure.

/api/delete-user is broken in production

It lives at `src/pages/api/`, which Vercel does not serve, so deleting a user 404s. Left alone deliberately: the current handler accepts a uid with no authorisation check at all, so moving it to `api/` would publish that. It needs an admin-of-the-same-school check written first, which is a separate piece of work.

Known gap, unchanged. Guardian sessions in the parent app are anonymous, so a payment is bound to the student it names rather than a verified family — the KNOWN GAP at the foot of `firestore.rules`. The exposure is narrow and one-way: the worst an unrelated session can do is pay its own money into a child's fee account. It closes when parents move to Firebase Phone Auth and the number is in the token.

Verified before handover: web build clean, ESLint unchanged from baseline (no new issues), 100/100 Firestore rules tests passing, flutter analyze 14 issues — all pre-existing, none in the new files.